

FACULTY PROFILE



PARTICULARS	DETAILS	
Name	ASHUTOSH CHANDRA BHENSLE	
Designation	LECTURER	
Branch	Computer Science & Engg.	
Date Of Birth	09/10/1985	
AICTE Unique Id	1-7330526191	
Education Qualification	M.Tech (Computer Science & Engineering)	
Work Experience:	Teaching	16 YEARS
	Research	NIL
	Industry	NIL
	Others	NIL
Area of Specialization	Computer Science	
Courses taught (Please Tick)	Diploma	YES
	Post Diploma	NIL
	Under Graduate	YES
	Post Graduate	NIL
	PostGraduate Diploma	NIL
Research guidance (No. of Students)	Master	NIL
	PhD-	NIL
No. of papers published in	National Journals	NIL
	International Journal	02
	Conferences	01
Master (Completed /Ongoing)	COMPLETED	
PhD (Completed/Ongoing)	ONGOING	
Projects Carried Out	NIL	
Patents (Filed & Granted)	NIL	
Technology Transfer	NIL	
Research Publications (No. of papers published in National/International/ Journals/Conferences)	03	
No.of Books Published with Details(Name of Book, Publisher with ISBN, Year of Publication etc.)	NIL	

Govt Girls Polytechnic Raipur
Department of Computer Sc. Engg
Semester -3rd
Subject – Data Structure
Subject code- 2022373(022)
Lecture Notes
Prepared By – A. C. Bhensle

Unit -1: Introction to data structure

A **data structure** is a specific way of organizing and storing data in a computer so that it can be used efficiently, allowing for effective processing, retrieval, and management of information. It combines primitive data types like integers and characters into a cohesive format that enables higher-level operations such as sorting, searching, insertion, and deletion.

Why are Data Structures Important?

Data structures are fundamental to programming and computer science for several reasons:

- **Efficiency:** They help manage large, complex sets of information efficiently, optimizing the use of CPU time and memory space (known as time and space complexity).
- **Problem Solving:** Choosing the correct data structure is crucial for designing efficient algorithms and solving complex problems in various applications, from operating systems to search engines and machine learning.
- **Organization:** They provide a logical and systematic way to store and retrieve data, making code simpler to understand and maintain.

Types of Data Structures

Data structures are generally classified into two main categories:

Linear Data Structures

In linear data structures, elements are arranged in a sequential, one-dimensional order, where each item is connected to the next and previous adjacent elements.

- **Arrays:** A collection of data items of the same type stored at contiguous memory locations, allowing for easy access using an index.
- **Linked Lists:** Elements (nodes) are stored in a linear order but are not in contiguous memory locations; instead, each node points to the next node using a pointer. This makes insertions and deletions efficient.
- **Stacks:** Follows the **Last In, First Out (LIFO)** principle, meaning the last element added is the first one to be removed (like a stack of plates).
- **Queues:** Follows the **First In, First Out (FIFO)** principle, meaning the first element added is the first one to be removed (like a line at a ticket counter).

Non-Linear Data Structures

In non-linear data structures, data elements are not arranged sequentially and can be connected to two or more other items, often in a hierarchical or networked structure.

- **Trees:** A hierarchical data structure with a single parent node (the root) and child subnodes branching out. They are often used to represent hierarchies like file systems or organizational maps.
- **Graphs:** Consist of a collection of nodes (vertices) connected by edges, which represent relationships between objects. They are used in modeling social networks, transportation networks, and finding the shortest path in GPS systems.
- **Hash Tables (or Maps):** Use a hash function to map keys to values for very fast data retrieval, addition, and removal, often used for databases and dictionaries.

Array:

One-dimensional arrays in C are declared by specifying the data type, name, and size (e.g., `int arr[5];`) and initialized using curly braces `{}`. Operations include traversing, inserting elements at specific indices, and searching (linear or binary). Key syntax includes `arr[index]` for access, allowing memory-efficient storage and fast, index-based data handling.

1. Declaring and Initializing One-Dimensional Array

- **Declaration:** Defines the type and size, allocating contiguous memory.
 - *Syntax:* `data_type array_name[size];`
 - *Example:* `int marks[5];` (Creates an array of 5 integers)
- **Initialization:** Assigns values to elements.
 - *At Declaration:* `int arr[5] = {10, 20, 30, 40, 50};`
 - *Partial Initialization:* `int arr[5] = {1, 2};` (Remaining elements become 0)
 - *Without Size:* `int arr[] = {1, 2, 3};` (Size is 3)

2. Array Operations

A. Insertion

Inserting an element at a specific position requires shifting subsequent elements.

- **Example (Insert at index 2):**

```
c
int arr[6] = {10, 20, 30, 40, 50}; // Initial size 5, room for 6
int pos = 2, val = 25;
for (int i = 5; i > pos; i--) {
    arr[i] = arr[i - 1]; // Shift elements
}
arr[pos] = val; // Insert
```

B. Searching

Finding the position of a specific value.

- **Linear Search:** Checks each element sequentially.

```
c
int search(int arr[], int n, int key) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == key) return i; // Found
    }
    return -1; // Not found
}
```

- **Binary Search:** Requires sorted arrays, divides the search interval in half.

3. Accessing Elements

Array elements are indexed from

00

0

to

size-1 size minus 1

size-1

.

- *Syntax:* array_name[index]
- *Example:* int x = arr[0]; (Accesses the first element)

C Code Example: Operations

```
c
#include <stdio.h>
int main() {
    int arr[10] = {1, 2, 4, 5}; // Initialization
    int n = 4, val = 3, pos = 2; // Insert 3 at index 2

    // Insertion
    for (int i = n; i > pos; i--)
        arr[i] = arr[i - 1];
    arr[pos] = val;
    n++;

    // Searching
    int key = 3, found = -1;
    for (int i = 0; i < n; i++) {
        if (arr[i] == key) {
            found = i;
            break;
        }
    }
    printf("Value %d found at index: %d", key, found);
    return 0;
}
```

Multi-dimensional arrays are data structures with more than one dimension, commonly used to represent matrices (2D) or tensors (3D+) for tabular data and linear algebra. They store homogeneous data in continuous memory, with 2D arrays acting as tables of rows and columns, accessible via indices `[row][col]`. Core operations include transposition, addition, scalar multiplication, and matrix multiplication.

Multi-Dimensional Array Structure

- **2D Array (Matrix):** Represented as `type array[row][col]`, organized by rows and columns.
- **3D Array:** An array of 2D matrices, often used for volumes or pages of data.
- **Memory Representation:** Elements are stored in contiguous memory locations, usually in row-major order.

The main drawbacks of linear arrays stem from their fundamental design: a **fixed size** and the requirement for **contiguous memory allocation**. These characteristics lead to inefficiencies in memory management and certain common operations.

Key disadvantages include:

- **Fixed Size Limitation:** Once a traditional static array is created, its size cannot be changed. If the array runs out of space, a new, larger array must be created and all existing elements copied over, which is a time-consuming and memory-intensive process (an $O(n)$ operation). This can also lead to memory wastage if the allocated size is larger than required.
- **Inefficient Insertions and Deletions:** Adding or removing elements, especially from the middle of the array, is inefficient. All subsequent elements must be shifted to make room for a new element or to fill the gap left by a deleted one. This "shifting" process has a time complexity of $O(n)$, where 'n' is the number of elements.
- **Homogeneous Data Types (in some languages):** Arrays in many traditional programming languages like C and Java are designed to store elements of a single, uniform data type. This can be a limitation when needing to store a mix of different data types in a single collection.
- **Memory Overhead for Complex Structures:** While arrays themselves are memory efficient for simple data, they can be insufficient for complex data structures that require dynamic relationships between elements, such as trees or graphs. Other structures like linked lists require extra memory for pointers, but offer more flexibility in dynamic scenarios.

- **Manual Index Management:** Programmers must manually manage indices, increasing the risk of errors such as "index out of bounds" exceptions (especially in languages like C that do not perform automatic boundary checks). This can make programming more complex and error-prone.
- **Memory Fragmentation Issues:** Dynamic resizing of arrays can lead to memory fragmentation, where free memory is broken into small, non-contiguous blocks, making it difficult to allocate a single large block when needed later.

An **array** is a general-purpose data structure that stores a collection of elements of the same data type in contiguous memory locations, accessible via an index. A **string** is a sequence of characters used to store and manipulate text. In many programming languages, strings are fundamentally implemented using character arrays.

The Character Array

A character array (`char[]` in C/C++/Java) is an array specifically designed to hold individual characters.

- **Declaration and Access:** You declare a character array by specifying the `char` data type and a name, often with a size or initial values. Elements are accessed using zero-based indexing (e.g., `charArray[0]`).
- **Mutability:** Character arrays are typically mutable, meaning their individual elements can be changed after creation. This makes them useful for tasks requiring in-place modification of character data.
- **Memory Management:** Elements are stored in contiguous memory locations.
- **C-style Strings:** In languages like C, strings are literally just character arrays that are terminated by a special **null character** (`\0`) to mark the end of the sequence.

The String (Data Type/Class)

In modern languages like Java and Python, `String` is a built-in, higher-level data type or class that provides more functionality than a basic character array.

- **Abstraction:** It abstracts away the low-level details of memory management and null termination.

- **Immutability (in Java/Python):** Strings in Java and Python are generally immutable; once a string object is created, its value cannot be changed. Any operation that appears to modify a string actually creates a new string object.
- **Built-in Functions:** The `String` class includes a rich set of built-in methods for common operations like finding substrings, concatenation (often using the `+` operator), and comparing strings, which simplifies coding.
- **Memory Management:** Strings may be stored in a dedicated "string constant pool" (in Java) for memory optimization, rather than the general heap.

Key Differences (e.g., in Java/C++)

Property	String	Character Array
Type	Object/Class (in Java)	Data structure / primitive type array
Mutability	Immutable (cannot be changed after creation)	Mutable (individual characters can be changed)
Access	Accessed using methods (e.g., <code>charAt()</code>)	Accessed using array indexing (<code>[]</code>)
Functions	Many built-in methods for manipulation	Few or no built-in methods; requires manual handling or utility libraries
Use Case	General text handling, API interaction	Low-level data manipulation, security-sensitive data (passwords)

UNIT 2 : STACK AND QUEUE

A **stack** is a linear data structure that follows the **Last-In, First-Out (LIFO)** principle, meaning the last element added is the first one to be removed. It operates like a stack of physical plates, where you add and remove plates only from the top.

Core Concepts

- **LIFO Principle:** This is the fundamental rule of a stack. The element that is inserted last is the first to be accessed and removed.
- **Top:** This is a pointer or index that keeps track of the most recently inserted element, as all operations (insertion and deletion) occur at this single end.
- **Abstract Data Type (ADT):** A stack is an ADT, meaning its behavior is defined by its operations, independent of how it is implemented (e.g., using arrays or linked lists).

Basic Operations

Stacks typically support the following primary operations, which generally have a time complexity of $O(1)$:

- **push()**: Inserts a new element onto the top of the stack. If the stack is full, an "overflow" error occurs.
- **pop()**: Removes and returns the top element from the stack. If the stack is empty, an "underflow" error occurs.
- **peek()** (or **top()**): Returns the value of the top element without removing it.
- **isEmpty()**: Checks if the stack is empty (returns `true` or `false`).
- **isFull()**: Checks if the stack has reached its maximum capacity (only relevant for fixed-size implementations like arrays).

Implementation Methods

1. Array Implementation

In an array-based implementation, a fixed-size array is used to store elements, and a variable (commonly named `top`) tracks the index of the most recently inserted element.

- **Initialization:** The `top` pointer is initialized to -1 to indicate an empty stack.
- **push:** Increments `top` and places the new element at `stack[top]`. Checks for "overflow" if the array is full.
- **pop:** Checks for "underflow" if `top` is -1. Otherwise, it retrieves the element at `stack[top]` and decrements `top`.

- **Advantages:** Simple to implement and memory-efficient as no extra space is needed for pointers.
- **Disadvantages:** Has a fixed size, which can lead to stack overflow if capacity is reached.

2. Linked List Implementation

A linked list implementation allows for a dynamic size stack that can grow and shrink as needed, avoiding the fixed-size limitation of arrays.

- **Initialization:** The `top` (or `head`) pointer is initialized to `NULL` (or `null`) to indicate an empty stack.
- **push:** A new node is created and inserted at the beginning of the linked list. The new node's `next` pointer is set to the current `top`, and `top` is updated to point to the new node.
- **pop:** Checks for "underflow" if `top` is `NULL`. Otherwise, a temporary pointer stores the top node, `top` is moved to the next node, and the memory for the old top node is freed.
- **Advantages:** Dynamic memory management and reduced risk of overflow (only limited by total system memory).
- **Disadvantages:** Requires more memory per element due to the storage of pointers in each node.

Real-World Applications

Stacks are fundamental to many computer science applications:

- **Function Call Management:** Programming languages use a "call stack" to manage function calls and their local variables; the last function called must finish first.
- **Undo/Redo Mechanisms:** Software editors store a history of user actions in a stack, allowing the last action performed to be the first one reversed.
- **Web Browser History:** The back button in a web browser uses a stack to navigate to previously visited pages in reverse order of visit.
- **Expression Evaluation:** Compilers and calculators use stacks to parse and evaluate mathematical expressions, such as converting infix to postfix notation.
- **Syntax Checking:** Compilers use stacks to ensure that parentheses, brackets, and braces are balanced and properly nested in code.

- **Backtracking Algorithms:** Stacks help in algorithms for solving problems like mazes by storing previous states to return to in case of a dead end.

Expression evaluation

Expression evaluation using a stack is a fundamental concept in computer science, used by compilers and calculators to process arithmetic and logical expressions efficiently. Stacks are particularly useful because their Last-In, First-Out (LIFO) nature helps manage operator precedence and the order of operands.

Stacks are used to evaluate expressions in all three major notations:

- **Infix Notation:** The operator is between the operands (e.g., $A + B$). This is standard human notation.
- **Postfix Notation (Reverse Polish Notation):** The operator is after the operands (e.g., $A B +$).
- **Prefix Notation (Polish Notation):** The operator is before the operands (e.g., $+ A B$).

Evaluating Postfix Expressions

Postfix is the most straightforward notation to evaluate using a stack because it doesn't require parentheses or complex precedence rules; the order of operations is determined by the placement of the operators.

The algorithm involves a single operand stack:

1. **Initialize an empty stack** for operands.
2. **Scan the expression from left to right**, character by character.
 1. **If the character is an operand** (a number or variable), push it onto the stack.
 2. **If the character is an operator** (e.g., $+$, $-$, $*$, $/$), pop the top two operands from the stack (let the first popped be $op2$ and the second popped be $op1$).
 3. **Perform the operation** using the format $op1 \text{ operator } op2$.
 4. **Push the result** of the operation back onto the stack.

Continue until the entire expression has been scanned.

The final result will be the only value remaining on the stack.

**Example: Evaluate "5 3 + 8 2 -"*

Scanned Token	Stack (bottom to top)	Explanation
5	[5]	Operand: push onto stack
3	[5, 3]	Operand: push onto stack
+	[8]	Operator: pop 3 and 5, calculate $5 + 3 = 8$, push result
8	[8, 8]	Operand: push onto stack
2	[8, 8, 2]	Operand: push onto stack
-	[8, 6]	Operator: pop 2 and 8, calculate $8 - 2 = 6$, push result
*	[48]	Operator: pop 6 and 8, calculate $8 * 6 = 48$, push result

The result is **48**.

Evaluating Infix Expressions

Evaluating an infix expression directly requires handling operator precedence and parentheses, which makes it more complex. It typically involves using **two stacks**: one for operands and one for operators.

The general approach is to iterate through the expression and, based on precedence rules and parentheses, move operators to a stack, perform operations when a lower-precedence operator is encountered, and manage a separate stack for numerical values.

Often, the most efficient method to evaluate an infix expression is to first convert it into postfix notation using a stack-based algorithm (known as the Shunting-yard algorithm concept), and then evaluate the resulting postfix expression using the simpler single-stack method described above.

Infix to Postfix

Infix notation is the standard mathematical format where operators are placed between operands (e.g.,

$A+B$ cap A plus cap B
 $A+B$

). Postfix notation, or **Reverse Polish Notation (RPN)**, places the operator after its operands (e.g.,

AB+cap A cap B plus

AB+

), eliminating the need for parentheses and simplifying computer processing.

Operator Precedence and Associativity

To convert correctly, you must follow the hierarchy of operators:

Operator	Description	Precedence	Associativity
^	Exponentiation	3 (Highest)	Right to Left
*, /	Multiplication, Division	2	Left to Right
+, -	Addition, Subtraction	1 (Lowest)	Left to Right

Step-by-Step Conversion Algorithm

Using a **Stack** is the most efficient way to manage this conversion.

1. **Scan from Left to Right:** Read the infix expression token by token.
2. **Operand:** If the token is an operand (like

A,B,1,2cap A comma cap B comma 1 comma 2

A,,1,2

), add it directly to the **Output String**.

3. **Left Parenthesis (:** Push it onto the **Stack**.
4. **Right Parenthesis):** Pop from the stack and add to the output until a (is reached. Discard the (.
5. **Operator:**
 - o If the stack is empty or the top is (, push the operator.
 - o If the current operator has **higher precedence** than the one on top, push it.
 - o If the current operator has **lower or equal precedence**, pop operators from the stack to the output until you find one with lower precedence (or the stack is empty), then push the current operator.
6. **End of Expression:** After scanning everything, pop all remaining operators from the stack to the output.

Example Walkthrough

Infix Expression: A + B * (C ^ D - E)

Token	Action	Stack	Output (Postfix)
A	Add to Output	Empty	A
+	Push to Stack	+	A

B	Add to Output	+	AB
*	Push (higher precedence than +)	+ *	AB
(	Push	+ * (	AB
C	Add to Output	+ * (	ABC
^	Push	+ * (^	ABC
D	Add to Output	+ * (^	ABCD
-	Pop ^ (higher), then Push -	+ * (-	ABCD^
E	Add to Output	+ * (-	ABCD^E
)	Pop until (	+ *	ABCD^E-
End	Pop remaining	Empty	ABCD^E-*+

Final Postfix: ABCD^E-*+

Infix to Prefix

convert an **infix** expression (like $A + B$) to a **prefix** expression (like $+ A B$), the most efficient standard method involves using a stack to manage operator precedence.

The Stack-Based Algorithm

This process effectively turns the infix expression into a "reversed" postfix expression.

1. **Reverse the Infix Expression:** Flip the string from end to beginning.
2. **Modify Parentheses:** Change every (to) and every) to (in the reversed string.
3. **Perform "Modified" Postfix Conversion:** Scan the reversed string from left to right:
 - **Operands:** Add them directly to the output.
 - **Right Parenthesis) :** Push it onto the stack.
 - **Left Parenthesis (:** Pop from the stack and add to the output until a) is encountered.
 - **Operators:**
 - If the incoming operator has **higher precedence** than the stack's top, push it.
 - If it has **equal precedence**, use associativity (for ^, pop it; for others, push it).
 - If it has **lower precedence**, pop the stack to the output until the condition is met.

Final Reversal: Reverse the resulting output string to get the final prefix notation.

Step-by-Step Example

Infix Expression: $(A - B / C) * (A / K - L)$

1. **Reverse & Swap Parentheses:** $(L - K / A) * (C / B - A)$

2. **Process via Stack (Mental or Code):**

- $L - K / A$ becomes $L K A / -$ in postfix.
- $C / B - A$ becomes $C B / A -$ in postfix.
- Combined with $*$: $L K A / - C B / A - *$

Reverse Final Result: $* - A / B C - / A K L$.

Operator Precedence Table

Operator	Description	Precedence	Associativity
$^$	Exponentiation	3 (Highest)	Right-to-Left
$*, /$	Multi/Div	2	Left-to-Right
$+, -$	Add/Sub	1 (Lowest)	Left-to-Right

Recursion

Recursion is a programming technique in which a function solves a problem by calling a smaller instance of itself. It is a powerful problem-solving approach frequently used in data structures and algorithms, particularly for problems that can be broken down into smaller, similar subproblems.

Core Principles of Recursion

A recursive function must have two main components to function correctly and prevent infinite loops:

- **Base Case:** This is the stopping condition. It is the simplest instance of the problem that can be solved directly without further recursion. When the base case is met, the function stops calling itself, and the results begin to unwind back up the chain of calls.

- **Recursive Case (or Recurrence Relation):** This is the part where the function calls itself with a modified, smaller input. Each recursive call must move the problem closer to the base case.

How Recursion Works Internally

Recursion is implemented using the **call stack**, a fundamental data structure that follows the Last-In, First-Out (LIFO) principle.

1. **Stack Frames:** Each time a function (including a recursive one) is called, an **activation record** or **stack frame** is created and pushed onto the call stack. This frame stores the function's local variables, parameters, and the return address (where the program should resume after the function finishes).
2. **Function Calls:** As the function recursively calls itself, new stack frames are continuously added on top of the previous ones.
3. **Unwinding the Stack:** When the base case is reached, the function returns a value. Its stack frame is popped off the stack, and control returns to the previous function call, which then completes its task and is popped in turn. This process continues until the initial function call is completed.

Key Applications in Data Structures

Recursion provides elegant solutions for problems involving data structures that have an inherent recursive nature. Common applications include:

- **Tree Traversals:** Algorithms like inorder, preorder, and postorder traversals of binary trees are naturally recursive.
- **Graph Traversals:** Depth-First Search (DFS) algorithms use recursion to explore graph nodes.
- **Divide and Conquer Algorithms:** Sorting algorithms such as Merge Sort and Quick Sort use recursion to break the list into smaller subproblems.
- **Backtracking:** Many complex problems, like solving a Sudoku puzzle or the N-Queens problem, are solved using backtracking algorithms, which are typically implemented recursively.
- **Towers of Hanoi:** This classic puzzle is a prime example of a problem with a simple, elegant recursive solution.

queue

A **queue** is a linear data structure that operates on the **First In, First Out (FIFO)** principle. This means the first element added to the queue is the first one to be removed, much like a line of people waiting for a service in real life.

Key Concepts

- **Linear Data Structure:** Elements are arranged in a sequential order.
- **FIFO Principle:** The rule that determines the order of processing. The element that has been in the queue the longest is processed first.
- **Two Ends:** A queue has two distinct ends:
 - **Front (or Head):** The end from which elements are removed (deletion).
 - **Rear (or Tail):** The end where new elements are added (insertion).

Basic Operations

The primary operations performed on a queue are:

- **Enqueue:** The process of adding a new element to the rear of the queue.
- **Dequeue:** The process of removing an element from the front of the queue.
- **Peek (or Front/Top):** Retrieves the value of the front element without removing it.
- **isEmpty:** Checks if the queue has any elements.
- **isFull:** Checks if the queue has reached its maximum capacity (relevant in array-based implementations).

Implementations and Types

Queues can be implemented using various methods, including arrays and linked lists.

There are also different types of queues designed for specific scenarios:

- **Circular Queue:** An array-based queue where the rear connects back to the front, allowing for more efficient use of memory by reusing dequeued spaces.
- **Priority Queue:** Each element has a priority, and the element with the highest priority is dequeued first, regardless of the insertion order.
- **Double-Ended Queue (Deque):** Allows insertion and deletion from both the front and rear ends, making it versatile for different uses.

Applications

Queues are fundamental in computer science and have numerous real-world applications:

- **Printer Job Scheduling:** Multiple print requests are stored in a queue and processed in the order they were received.
- **CPU Scheduling:** Operating systems use queues to manage processes waiting for CPU time.
- **Keyboard Buffering:** Keystrokes are temporarily held in a queue-like buffer to ensure they are processed and displayed in the correct order.
- **Breadth-First Search (BFS):** A graph traversal algorithm that uses a queue to explore nodes level by level.
- **Web Servers:** Handling requests in the order they arrive to ensure fair and orderly service.

Implementation of Queue Using Array

Here is the implementation of the queue using array in C, C++, and Java languages:

C Program for Queue Using Array

Below is a simple C program to implement a queue using an array.

```
#include <stdio.h>

#include <stdlib.h>

#define MAX 5
```

```

struct Queue {

    int items[MAX];

    int front, rear;

};

void initQueue(struct Queue* q) {

    q->front = -1;

    q->rear = -1;

}

int isFull(struct Queue* q) {

    return q->rear == MAX - 1;

}

int isEmpty(struct Queue* q) {

    return q->front == -1;

}

void enqueue(struct Queue* q, int value) {

    if (isFull(q)) {

        printf("Queue is full\n");

        return;
    }

```

```

    }

    if (q->front == -1) {

        q->front = 0;

    }

    q->rear++;

    q->items[q->rear] = value;

    printf("Enqueued: %d\n", value);
}

int dequeue(struct Queue* q) {

    if (isEmpty(q)) {

        printf("Queue is empty\n");

        return -1;

    }

    int item = q->items[q->front];

    if (q->front == q->rear) {

        q->front = q->rear = -1;

    } else {

        q->front++;

    }

    return item;

}

```

```

int main() {

    struct Queue q;

    initQueue(&q);

    enqueue(&q, 10);

    enqueue(&q, 20);

    enqueue(&q, 30);

    printf("Dequeued: %d\n", dequeue(&q));

    return 0;

}

```

UNIT 3 Pointers

A **pointer** in C is a variable that stores the **memory address** of another variable. They allow for direct memory manipulation, which is crucial for efficient programming, dynamic memory allocation, and building complex data structures like linked lists and trees.

Key Concepts

- **Address-of Operator (&):** This unary operator returns the memory address of a variable.
- **Dereference Operator (*):** Also known as the indirection operator, it is used to access or modify the value stored at the memory address the pointer points to.
- **Declaration Syntax:** Pointers are declared by specifying the data type of the variable they will point to, followed by an asterisk (*) and the pointer name.

Usage and Syntax

Here is a simple example demonstrating how to declare, initialize, and use a pointer in C:

C

```
#include <stdio.h>

int main() {
    int num = 10;          // A normal integer variable
    int *ptr;              // A pointer variable 'ptr' that can point to an
                           // integer

    ptr = &num;            // Assign the address of 'num' to 'ptr'

    printf("Value of num: %d\n", num);          // Direct access: 10
    printf("Address of num: %p\n", &num);       // Address using & operator
    printf("Value of ptr (address): %p\n", ptr); // Pointer value (same
address)
    printf("Value at address pointed by ptr: %d\n", *ptr); // Dereference: 10

    *ptr = 20;             // Change the value of 'num' through the pointer
    printf("Updated value of num: %d\n", num);  // num is now 20

    return 0;
}
```

- `int *ptr;`: This declares `ptr` as a pointer to an `int` type.
- `ptr = #`: The `&` operator gets the address of `num` and stores it in `ptr`.
- `*ptr`: This expression accesses the data stored at the address contained in `ptr`.

Types of Pointers

C supports several types of pointers:

- **Null Pointer:** A pointer that does not point to any valid memory location, typically assigned `NULL` (or `nullptr` in C23) to avoid errors.
- **Void Pointer:** A generic pointer that can hold the address of any data type. It must be typecast before being dereferenced because the compiler does not know the size of the data it points to.

- **Wild Pointer:** An uninitialized pointer that points to a random memory location, leading to potential crashes or undefined behavior.
- **Dangling Pointer:** A pointer that points to a memory location that has been freed or deallocated, but the pointer itself hasn't been updated.
- **Pointer to Pointer (Double Pointer):** A pointer variable that stores the address of another pointer.

Applications

Pointers are a fundamental and powerful aspect of C programming:

- **Dynamic Memory Management:** Functions like `malloc()`, `calloc()`, `realloc()`, and `free()` use pointers to allocate and deallocate memory at runtime.
- **Efficient Array and Structure Handling:** Compilers often use pointers internally to access array elements, and pointers provide efficient ways to handle large data structures.
- **Passing Arguments by Reference:** Pointers allow a function to modify the actual value of a variable in the calling function (pass-by-reference), rather than just a local copy (pass-by-value).
- **Returning Multiple Values:** Functions can use pointers passed as arguments to return multiple modified values to the caller.
- **Data Structures:** Pointers are essential for implementing complex data structures such as linked lists, queues, trees, and graphs.

Pointer Arithmetic

Pointer arithmetic is a key feature that allows moving a pointer through memory based on the size of the data type it points to. When an integer value is added to or subtracted from a pointer, the address is incremented/decremented by the integer value multiplied by the `sizeof()` the data type.

- **Increment (++) and Decrement (--):** These operators increase or decrease the pointer's address to point to the *next* or *previous* element of its type, respectively.
- **Addition of an Integer (+):** Adding an integer `n` to a pointer `ptr` (e.g., `ptr + n`) results in a new address that is `n * sizeof(data_type)` bytes away from the original address. This is commonly used to access array elements (e.g., `*(ptr + i)` is equivalent to `array[i]`).

- **Subtraction of an Integer (-):** Subtracting an integer `n` from a pointer `ptr` (e.g., `ptr - n`) moves the pointer backward by `n` elements.
- **Subtraction of Two Pointers (-):** Subtracting two pointers of the same type (e.g., `ptr1 - ptr2`) yields an integer value representing the **number of elements** between the two memory addresses, not the raw address difference in bytes. This is only valid if both pointers point to elements within the same array or memory block.

Pointer Comparison

Pointers can be compared using relational operators (`==`, `!=`, `<`, `>`, `<=`, `>=`) only if they point to the same data type or to elements of the same array. The comparison checks whether the addresses they hold are equal, unequal, or ordered in memory.

Illegal Operations

Certain arithmetic operations are not allowed on pointers because they do not produce meaningful or valid memory addresses:

- Addition of two pointers (e.g., `ptr1 + ptr2`)
- Multiplication of pointers (e.g., `ptr1 * ptr2`)
- Division of a pointer (e.g., `ptr1 / 2` or `ptr1 / ptr2`)
- Modulus or bitwise operations on pointers.

An **array of pointers** is a collection of pointer variables stored in contiguous memory locations, where each element (pointer) stores the memory address of another variable or data structure. This powerful construct provides flexibility, especially when managing data of varying sizes, such as strings of different lengths.

Declaration and Initialization

The syntax for declaring an array of pointers is:

```
C
data_type *array_name[size];
```

- `data_type`: The type of data the pointers will point to.
- `*`: The indirection operator, indicating an array of pointers.
- `array_name`: The name of the array.
- `size`: The number of pointers in the array.

For example, to declare an array of 3 pointers to integers:

C

```
int *ptr_array[3];
```

You can initialize the array at the time of declaration using the address-of operator (&):

C

```
int v1 = 10, v2 = 20, v3 = 30;
int *ptr_array[3] = {&v1, &v2, &v3};
```

Each element `ptr_array[i]` holds the address of a specific integer variable.

Accessing Values

To access the value pointed to by a pointer in the array, you must first access the pointer using its index and then dereference it using the asterisk (*) operator.

C

```
// Accessing the value of v2 using the array of pointers
printf("Value of v2: %d\n", *ptr_array[1]); // Output: Value of v2: 20
```

pointer to a pointer

A **pointer to a pointer**, also known as a **double pointer**, is a variable in C that stores the memory address of another pointer variable. This creates multiple levels of indirection to the final data value.

Declaration and Initialization

A double pointer is declared using two asterisks (**).

C

```
int num = 10;           // An integer variable
int *ptr = &num;        // A pointer to an integer, storing the address of num
int **dptr = &ptr;      // A pointer to a pointer, storing the address of ptr
```

In this example:

- `num` holds the value 10.
- `ptr` holds the address of `num`.
- `dptr` holds the address of `ptr`.

Accessing Values (Dereferencing)

To access the final value through a double pointer, you must dereference it twice using the asterisk (*).

- `*dptr` gives the value stored at the address `dptr` points to, which is the address of `num` (the value of `ptr`).
- `**dptr` dereferences it a second time to get the value stored at the address that `ptr` points to, which is the value of `num` (10).

Passing an array to function

In C, when an array is passed to a function, it is treated as a **pointer to its first element** (often referred to as "pass-by-reference" behavior). This means the function can modify the original array, and a copy of the entire array is not created. You should also pass the array's size as a separate argument to process all elements correctly.

Passing a One-Dimensional Array

You can declare the function parameter in one of three equivalent ways:

1. **As an unsized array:** `void myFunction(int param[])`
2. **As a sized array:** `void myFunction(int param[10])` (the size is ignored by the compiler)
3. **As a pointer:** `void myFunction(int *param)`

The function receives a pointer in all cases. The third method (using a pointer) is often the most flexible.

Here is a complete example demonstrating the most common and recommended way to pass an array and its size:

C

```
#include <stdio.h>

// Function declaration: takes a pointer to an integer (the array) and an
integer (the size)
void printArray(int *arr, int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    int numbers[] = {10, 20, 30, 40, 50};
```

```

// Calculate the size of the array
int length = sizeof(numbers) / sizeof(numbers[0]);

// Call the function, passing the array name and its size
printArray(numbers, length);

return 0;
}

```

Output:

```
10 20 30 40 50
```

A **structure** in C is a **user-defined data type** that groups logically related variables of possibly different data types under a single name. It is used to represent real-world entities, such as a record for a student or an employee, in an organized manner.

Defining a Structure

You define a structure using the `struct` keyword, followed by a tag name and the member variables enclosed in curly braces.

C

```

struct Student {
    char name[50];
    int roll_number;
    float gpa;
};

```

- `struct` is the required keyword.
- `Student` is the *structure tag* (name).
- `name`, `roll_number`, and `gpa` are the *members* (or fields) of the structure.

Note that defining a structure only creates a template; it does not allocate any memory.

Declaring Structure Variables

Memory is allocated only when you declare variables of the structure type. There are two primary ways to declare structure variables:

- **After the definition (recommended for clarity):**

C

```
struct Student s1, s2;
```

- **Along with the definition:**

C

```
struct Student {  
    // ... members ...  
} s1, s2;
```

You can use the `typedef` keyword to create an alias, which simplifies the declaration process by allowing you to omit the `struct` keyword later:

C

```
typedef struct {  
    char name[50];  
    int roll_number;  
    float gpa;  
} Student; // 'Student' is now an alias for the structure type
```

```
Student s1, s2; // Cleaner declaration
```

Accessing Members

To access the individual members of a structure, you use the **dot (.) operator** (also called the member access operator) with the structure variable:

C

```
#include <stdio.h>  
#include <string.h> // Required for strcpy  
  
int main() {  
    struct Student s1;  
  
    // Assigning values to members  
    strcpy(s1.name, "John Doe"); // Use strcpy for string/char arrays  
    s1.roll_number = 101;  
    s1.gpa = 3.5;  
  
    // Accessing and printing values  
    printf("Name: %s\n", s1.name);  
    printf("Roll Number: %d\n", s1.roll_number);  
    printf("GPA: %.2f\n", s1.gpa);  
}
```

```
    return 0;
}
```

Key Operations & Concepts

- **Initialization:** Members can be initialized at declaration using an initializer list `struct Student s1 = {"John Doe", 101, 3.5};` or individually using the dot operator.
- **Pointers to Structures:** You can create pointers to structures. When accessing members via a structure pointer, you use the **arrow (->) operator**.
- **Arrays of Structures:** You can create arrays where each element is a structure, useful for managing a list of records (e.g., `struct Student class[50];`).
- **Functions:** Structures can be passed to functions by value or by reference (pointer). Passing by reference is more efficient for large structures.
- **Nested Structures:** A structure can contain another structure as a member, allowing for hierarchical data organization.

Union

In C programming, a **union** is a user-defined data type that allows multiple members to share the same memory location. This means that only one member can hold a valid value at any given time. The key difference from a `struct` is how memory is allocated: a union's size is determined by its largest member.

Union Declaration

The general syntax for declaring a union is similar to a structure, using the `union` keyword:

C

```
union UnionName {
    // member declarations
    datatype member1;
    datatype member2;
    ...
};
```

- The `union` keyword is required.
- `UnionName` is an optional tag for the union type.

- The members inside the braces define the different data types that can be stored in the shared memory space.

You can declare variables of the union type in two main ways:

- **During declaration:**

C

```
union Data {
    int i;
    float f;
    char str[20];
} data_var1, data_var2; // Variables declared here
```

- **After declaration:**

C

```
union Data {
    int i;
    float f;
    char str[20];
};

// Variables declared later in the code
union Data data_var1;
union Data data_var2;
```

- **Using typedef (for cleaner syntax):**

C

```
typedef union {
    int i;
    float f;
    char str[20];
} Data; // Data becomes an alias for the union type

Data data_var1; // Variable declared using the alias
```

Union Initialization

You can initialize a union variable at the time of declaration.

- **C89 style:** Only the first member of the union can be initialized.

C

```
union Data {
    int i;
    float f;
};
```

```
union Data data_var = {10}; // Initializes the first member 'i' to 10
```

- **C99 style (using designated initializers):** You can explicitly specify which member to initialize using the . operator.

C

```
union Data {
    int i;
    float f;
};
```

```
// Initializes the 'f' member to 3.14
```

```
union Data data_var = {.f = 3.14};
```

Accessing and Assigning Values

After declaration, you can access and assign values to members using the dot (.) operator (or the arrow (->) operator if using a pointer).

Important: Assigning a value to one member makes that the active member, overwriting any previous value stored by another member in the shared memory.

C

```
#include <stdio.h>
#include <string.h>
```

```
union Value {
    int i;
    float f;
    char str[20];
};
```

```
int main() {
```

```

union Value val;

val.i = 10; // 'i' is the active member
printf("Integer: %d\n", val.i);

val.f = 3.14; // 'f' becomes the active member, overwrites 'i'
printf("Float: %.2f\n", val.f);

// The value of 'i' is now likely garbage/undefined
printf("Integer after float assignment (undefined behavior): %d\n",
val.i);

// Use strcpy for string/char array assignment
strcpy(val.str, "Hello"); // 'str' becomes the active member, overwrites
'f'
printf("String: %s\n", val.str);

return 0;
}

```

Array of structure

An **array of structures (AoS)** is a collection of structure variables stored in contiguous memory locations, where each element of the array is a complete structure containing its own members. This data layout allows you to store and manage multiple records of the same type efficiently. It is commonly used in programming languages like C and C++.

Key Concepts

- **Structure:** A user-defined data type that groups related data items of different data types under a single name. For example, a `Person` structure might have members for `name` (string) and `age` (integer).
- **Array:** A collection of elements of the *same* data type stored in contiguous memory locations.

- **Array of Structures:** An array where the data type of each element is a structure. This is ideal for scenarios where you need a list of complex data, such as a list of employees, students, or points in a coordinate system.

Declaration and Access (in C)

The general syntax for declaring a structure and an array of that structure in C is:

C

```
// Define the structure
struct StructureName {
    dataType member1;
    dataType member2;
    // ...
};
```

```
// Declare an array of structures
struct StructureName arrayName[arraySize];
```

You access individual members of a structure within the array using the array index and the dot (.) operator:

C

```
// Example: Accessing the 'name' of the first person in the array
arrayName[0].member1;
```

Example (in C)

This example demonstrates how to define a `Person` structure, create an array of `Person` structures, initialize its elements, and iterate through the array to print the data:

C

```
#include <stdio.h>
#include <string.h>

struct Person {
    char name[50];
    int age;
};

int main() {
    // Declare an array of 2 Person structures
    struct Person people[2];
```

```

// Initialize the first element
strcpy(people[0].name, "Alice");
people[0].age = 30;

// Initialize the second element
strcpy(people[1].name, "Bob");
people[1].age = 25;

// Iterate through the array and print the information
for (int i = 0; i < 2; i++) {
    printf("Person %d:\n", i + 1);
    printf("  Name: %s\n", people[i].name);
    printf("  Age: %d\n", people[i].age);
}

return 0;
}

```

Pointer to Structure

In C, a **pointer to a structure** is a variable that stores the memory address of a `struct` variable. This technique is crucial for efficient memory management, dynamic memory allocation, and building complex data structures like linked lists.

Declaration and Initialization

To declare a pointer to a structure, you use the `struct` keyword followed by the structure's name and an asterisk `*` before the pointer's name.

C

```

struct Person {
    char name[50];
    int age;
};

// Declare a structure variable
struct Person person1;

```

```
// Declare a pointer to the structure
struct Person *personPtr;

// Initialize the pointer with the address of the structure variable
personPtr = &person1;
```

Accessing Structure Members

There are two primary ways to access the members of a structure using a pointer:

- **Using the indirection operator (*) and the dot operator (.):** The parentheses are necessary because the dot operator has a higher precedence than the indirection operator.

C

```
(*personPtr).age = 30;
strcpy((*personPtr).name, "Atisha");
```

- **Using the arrow operator (->):** C provides this as a shorthand notation, which is more readable and intuitive. It automatically dereferences the pointer and accesses the member in one step.

C

```
personPtr->age = 30;
strcpy(personPtr->name, "Atisha");
```

Structure vs Union

Parameter	Structure	Union
Definition	A structure is a user-defined data type that groups different data types into a single entity.	A union is a user-defined data type that allows storing different data types at the same memory location.
Keyword	The keyword struct is used to define a structure	The keyword union is used to define a union

Parameter	Structure	Union
Size	The size is the sum of the sizes of all members, with padding if necessary.	The size is equal to the size of the largest member, with possible padding.
Memory Allocation	Each member within a structure is allocated unique storage area of location.	Memory allocated is shared by individual members of union.
Data Overlap	No data overlap as members are independent.	Full data overlap as members shares the same memory.
Accessing Members	Individual member can be accessed at a time.	Only one member can be accessed at a time.

Unit 4 Linked List

A **linked list** is a fundamental linear data structure that consists of a sequence of interconnected elements called **nodes**. Unlike arrays, linked list nodes are not stored in contiguous memory locations; instead, each node contains a piece of **data** and a **pointer** (or reference) to the next node in the sequence, forming a chain-like structure.

Structure and Terminology

- **Node:** The basic building block of a linked list, which typically contains a data field and one or two pointers to other nodes.
- **Head:** A pointer to the first node in the linked list. It serves as the entry point to the list.
- **Tail / Last Node:** The final node in a linear linked list. Its pointer to the next node contains a `NULL` reference to indicate the end of the list.

Types of Linked Lists

Linked lists come in various types to suit different needs:

- **Singly Linked List:** The simplest type, where each node has a pointer only to the *next* node. Traversal is possible in only one direction (forward).
- **Doubly Linked List:** Each node has two pointers: one to the *next* node and one to the *previous* node. This allows for traversal in both forward and backward directions.
- **Circular Linked List:** The last node's pointer points back to the first node (head), forming a continuous loop. This eliminates the `NULL` reference at the end of the list.

Key Operations

Common operations performed on a linked list include:

- **Traversal:** Accessing each node in the list in sequence, starting from the head.
- **Insertion:** Adding a new node to the list (at the beginning, end, or a specific position). This is an efficient operation compared to arrays as it only requires updating a few pointers.
- **Deletion:** Removing a node from the list (at the beginning, end, or a specific position).
- **Searching:** Finding a node with a specific value by traversing the list.
- **Sorting:** Arranging the nodes in a specific order (e.g., ascending/descending data values).

Advantages and Disadvantages

Advantages:

- **Dynamic Size:** Linked lists can grow or shrink in size during runtime, using memory efficiently as needed.
- **Efficient Insertions/Deletions:** Operations for adding or removing elements are generally faster than in arrays, as no elements need to be shifted in memory.
- **Implementation Flexibility:** Used to implement other essential data structures like stacks, queues, and graphs.

Disadvantages:

- **Sequential Access:** Elements can only be accessed sequentially starting from the head; random access by index (like in arrays) is not possible.

- **Extra Memory Overhead:** Each node requires additional memory to store pointers, which can be significant for lists of small data items.
- **Slower Traversal:** Due to non-contiguous memory storage, linked lists can have poor cache locality, potentially leading to slower processing times than arrays for certain operations.

Singly Linked List

1. Insert at the beginning

- Allocate memory for new node
- Store data
- Change next of new node to point to head
- Change head to point to recently created node

```
struct node *newNode;  
newNode = malloc(sizeof(struct node));  
newNode->data = 4;  
newNode->next = head;  
head = newNode;
```

2. Insert at the End

- Allocate memory for new node
- Store data
- Traverse to last node
- Change next of last node to recently created node

```
struct node *newNode;  
newNode = malloc(sizeof(struct node));
```

```

newNode->data = 4;
newNode->next = NULL;

struct node *temp = head;
while(temp->next != NULL){
    temp = temp->next;
}

temp->next = newNode;

```

3. Insert at the Middle

- Allocate memory and store data for new node
- Traverse to node just before the required position of new node
- Change next pointers to include new node in between

```

struct node *newNode;
newNode = malloc(sizeof(struct node));
newNode->data = 4;

struct node *temp = head;

for(int i=2; i < position; i++) {
    if(temp->next != NULL) {
        temp = temp->next;
    }
}

newNode->next = temp->next;
temp->next = newNode;

```

Delete from a Linked List

You can delete either from the beginning, end or from a particular position.

1. Delete from beginning

- Point head to the second node

```
head = head->next;
```

2. Delete from end

- Traverse to second last element
- Change its next pointer to null

```
struct node* temp = head;
while(temp->next->next!=NULL){
    temp = temp->next;
}
temp->next = NULL;
```

3. Delete from middle

- Traverse to element before the element to be deleted
- Change next pointers to exclude the node from the chain

```
for(int i=2; i< position; i++) {
    if(temp->next!=NULL) {
        temp = temp->next;
    }
}

temp->next = temp->next->next;
```


Search an Element on a Linked List

You can search an element on a linked list using a loop using the following steps. We are finding `item` on a linked list.

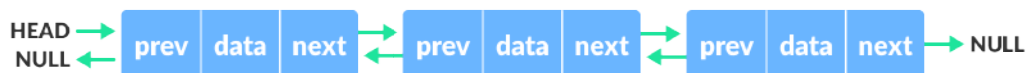
- Make `head` as the `current` node.
- Run a loop until the `current` node is `NULL` because the last element points to `NULL`.
- In each iteration, check if the key of the node is equal to `item`. If it the key matches the item, return `true` otherwise return `false`.

```
// Search a node
bool searchNode(struct Node** head_ref, int key) {
    struct Node* current = *head_ref;

    while (current != NULL) {
        if (current->data == key) return true;
        current = current->next;
    }
    return false;
}
```

Doubly Linked List

We add a pointer to the previous node in a doubly-linked list. Thus, we can go in either direction: forward or backward.



A node is represented as

```
struct node {
    int data;
```

```
    struct node *next;  
    struct node *prev;  
}
```

A three-member doubly linked list can be created as

```
/* Initialize nodes */  
struct node *head;  
struct node *one = NULL;  
struct node *two = NULL;  
struct node *three = NULL;  
  
/* Allocate memory */  
one = malloc(sizeof(struct node));  
two = malloc(sizeof(struct node));  
three = malloc(sizeof(struct node));  
  
/* Assign data values */  
one->data = 1;  
two->data = 2;  
three->data = 3;  
  
/* Connect nodes */  
one->next = two;  
one->prev = NULL;  
  
two->next = three;  
two->prev = one;  
  
three->next = NULL;  
three->prev = two;  
  
/* Save address of first node in head */  
head = one;
```

Circular Linked List

A circular linked list is a variation of a linked list in which the last element is linked to the first element. This forms a circular loop.



Circular linked list

A circular linked list can be either singly linked or doubly linked.

- for singly linked list, next pointer of last item points to the first item
- In the doubly linked list, prev pointer of the first item points to the last item as well.

A three-member circular singly linked list can be created as:

```
/* Initialize nodes */
struct node *head;
struct node *one = NULL;
struct node *two = NULL;
struct node *three = NULL;

/* Allocate memory */
one = malloc(sizeof(struct node));
two = malloc(sizeof(struct node));
three = malloc(sizeof(struct node));

/* Assign data values */
one->data = 1;
two->data = 2;
three->data = 3;

/* Connect nodes */
one->next = two;
two->next = three;
three->next = one;
```

```
/* Save address of first node in head */  
head = one;
```

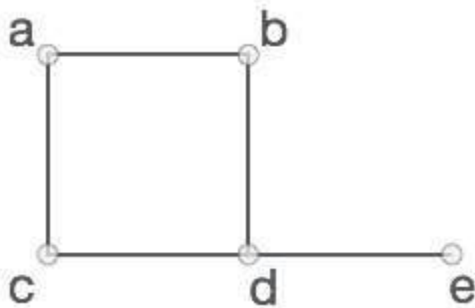
Unit 5: Graph and Tree

5.1 Definition of graph

What is a Graph?

A graph is an abstract data type (ADT) which consists of a set of objects that are connected to each other via links. The interconnected objects are represented by points termed as **vertices**, and the links that connect the vertices are called **edges**.

Formally, a graph is a pair of sets (V, E) , where V is the set of vertices and E is the set of edges, connecting the pairs of vertices. Take a look at the following graph –



In the above graph,

$$V = \{a, b, c, d, e\}$$
$$E = \{ab, ac, bd, cd, de\}$$

Representation of Graphs

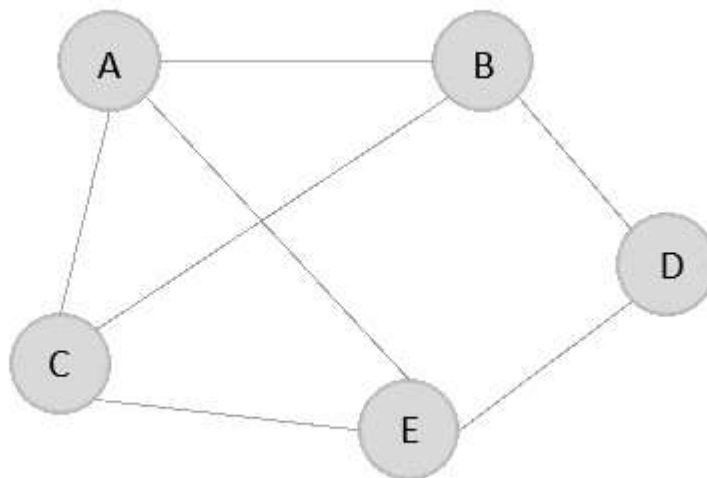
While representing graphs, we must carefully depict the elements (vertices and edges) present in the graph and the relationship between them. Pictorially, a graph is represented with a finite set of nodes and connecting links between them. However, we can also represent the graph in other most commonly used ways, like –

- Adjacency Matrix
- Adjacency List

Adjacency Matrix

The Adjacency Matrix is a $V \times V$ matrix where the values are filled with either 0 or 1. If the link exists between V_i and V_j , it is recorded 1; otherwise, 0.

For the given graph below, let us construct an adjacency matrix –

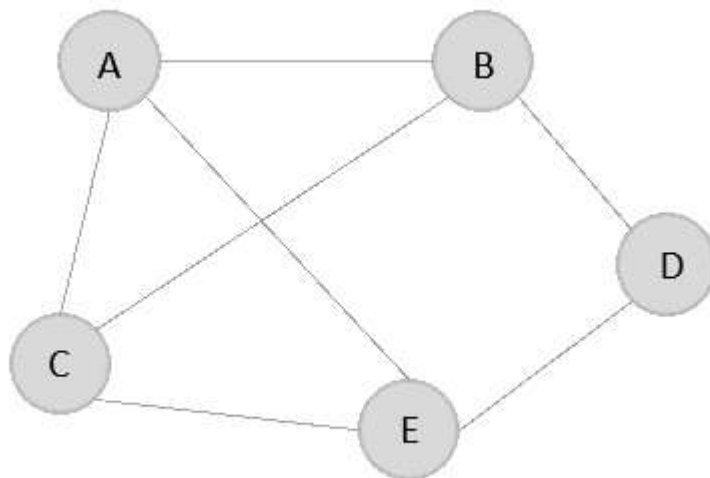


The adjacency matrix is –

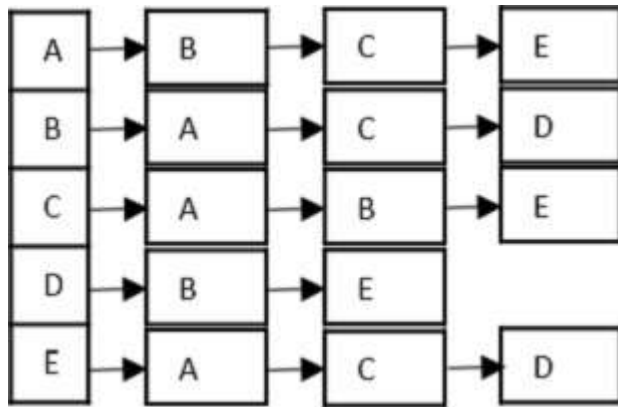
	A	B	C	D	E
A	0	1	1	0	1
B	1	0	1	1	0
C	1	1	0	0	1
D	0	1	0	0	1
E	1	0	1	1	0

Adjacency List

The adjacency list is a list of the vertices directly connected to the other vertices in the graph.



The adjacency list is –

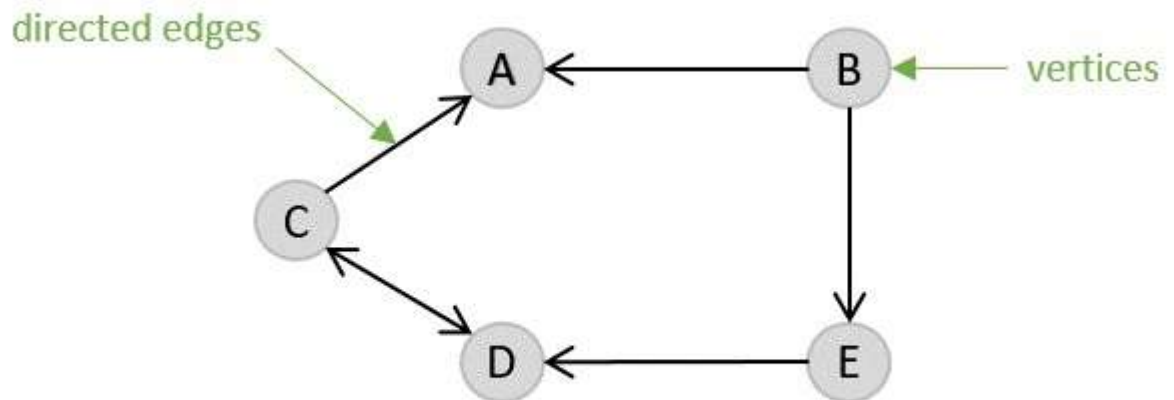


Types of graph

There are two basic types of graph –

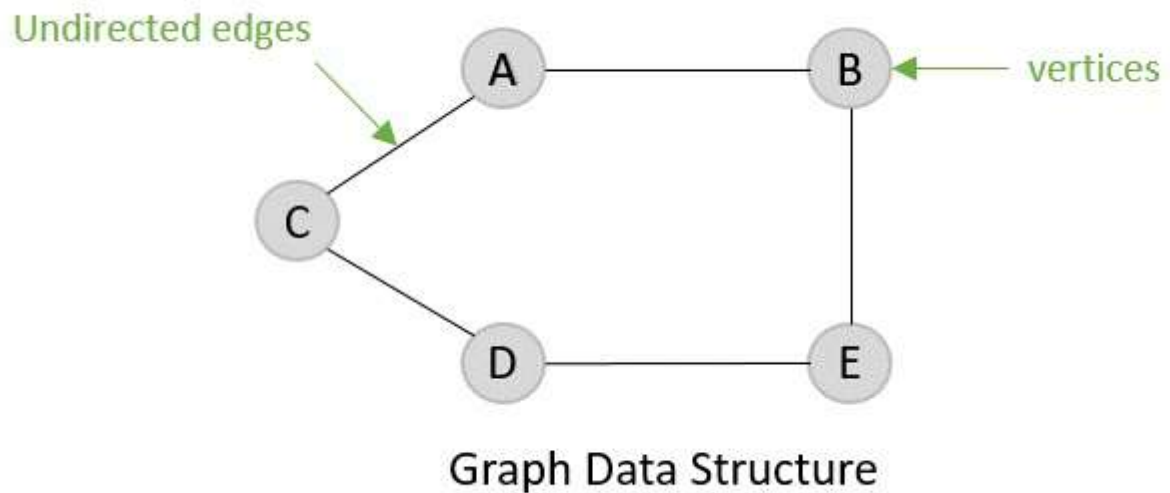
- Directed Graph
- Undirected Graph

Directed graph, as the name suggests, consists of edges that possess a direction that goes either away from a vertex or towards the vertex. Undirected graphs have edges that are not directed at all.



Graph Data Structure

Directed Graph



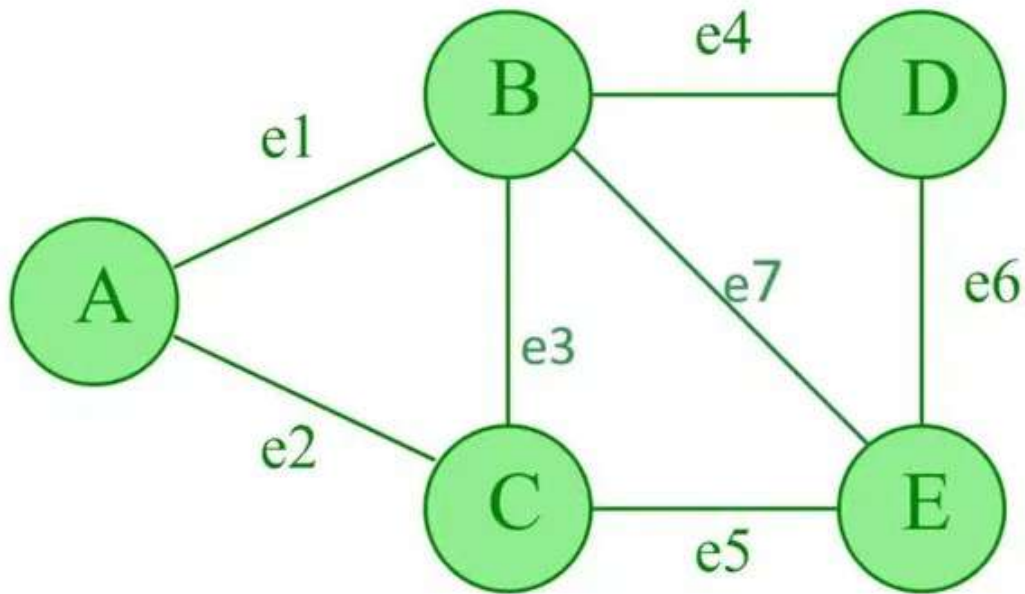
Undirected Graph

Incidence Matrix

The **incidence matrix** is a way of representing a graph where rows represent vertices and columns represent edges, each entry in the matrix indicates whether a vertex is incident to an edge. Unlike an adjacency matrix, which shows relationships between the vertices of a graph (whether there is an edge between two vertices), an incidence matrix represents the relationship between vertices and the edges of a graph (i.e. which vertices are connected by which edges). An incidence matrix is a $V \times E$ matrix, where V is the number of vertices and E is the number of edges in the graph.

Incidence Matrix for an Undirected Graph

Now, let us consider the following undirected graph to understand how the graph will be represented in the incidence matrix:



Undirected graph having 5 vertices and 7 edges

The incidence matrix for the above undirected graph is:

Node/Edges	e1	e2	e3	e4	e5	e6	e7
A	1	1	0	0	0	0	0
B	1	0	1	1	0	0	1
C	0	1	1	0	1	0	0
D	0	0	0	1	0	1	0
E	0	0	0	0	1	1	1

- In the above matrix, the columns represents the edge and the rows represents the vertices.
- If the value of the cell is 1 it indicates that the vertex is incident on the edge.

- If the value of the cell is 0 it indicates that the vertex is not incident on the edge.
- Like for the first column e1, the value for nodes A and B is 1 which indicates that there exists an edge between the nodes A and B in the graph. So, e1 connects A and B, e2 connects A and C, e3 connects B and C, e4 connects B and D, e5 connects C and E, e6 connects D and E, e7 connects B and E.

Tree

A **tree** is a non-linear, hierarchical data structure that organizes elements, called **nodes**, in a parent-child relationship. Unlike linear structures like arrays or linked lists, trees use **edges** (links) to connect nodes across multiple levels, which allows for efficient data organization, searching, and sorting.

Key Terminology

Understanding key terms is fundamental to working with trees:

- **Root Node:** The topmost node with no parent.
- **Edge:** The link between nodes.
- **Parent Node:** A node with children below it.
- **Child Node:** A node directly connected below a parent.
- **Leaf Node:** A node with no children.
- **Internal Node:** Any node with at least one child.
- **Sibling:** Nodes sharing the same parent.
- **Subtree:** A portion including a node and its descendants.
- **Height:** Number of edges on the longest path from a node to a leaf.
- **Depth:** Number of edges from the root to a node.

Why Use Trees?

Trees are used for hierarchical data and efficient operations. They model real-world hierarchies like file systems and offer efficient searching, insertion, and deletion in $O(\log n)$ time in balanced trees. Trees can also handle dynamic data.

Common Types of Trees

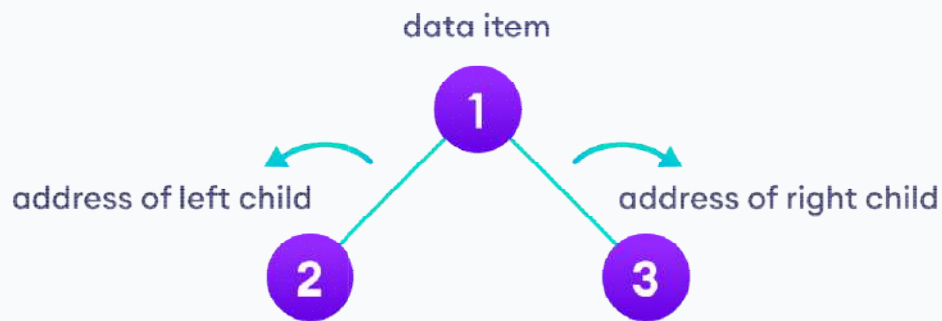
Different types are optimized for specific uses:

- **General Tree:** No limit on children per node.
- **Binary Tree:** At most two children per node.
- **Binary Search Tree (BST):** Binary tree where left subtree values are less than the node, and right values are greater.
- **Balanced BSTs (AVL, Red-Black):** Self-balancing BSTs for efficient $O(\log n)$ operations.
- **Heap:** Tree satisfying the heap property, used for priority queues.
- **B-Tree / B+ Tree:** Used in databases and file systems for large data blocks.

BINARY TREE

A binary tree is a tree data structure in which each parent node can have at most two children. Each node of a binary tree consists of three items:

- data item
- address of left child
- address of right child

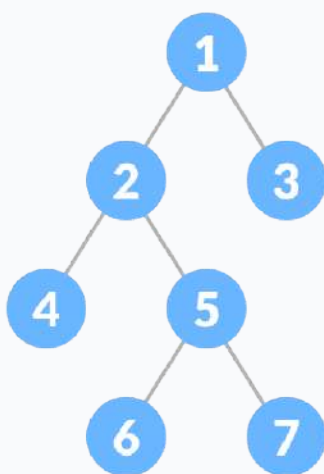


Binary Tree

TYPES OF BINARY TREE

1. FULL BINARY TREE

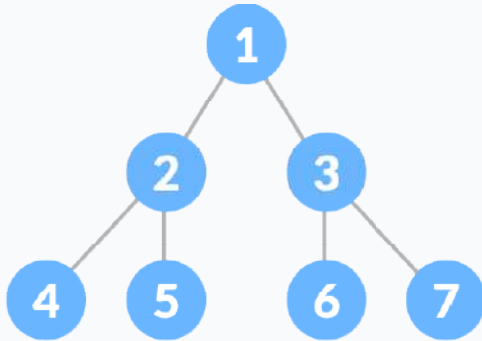
A full Binary tree is a special type of binary tree in which every parent node/internal node has either two or no children.



Full Binary Tree

2. PERFECT BINARY TREE

A perfect binary tree is a type of binary tree in which every internal node has exactly two child nodes and all the leaf nodes are at the same level.

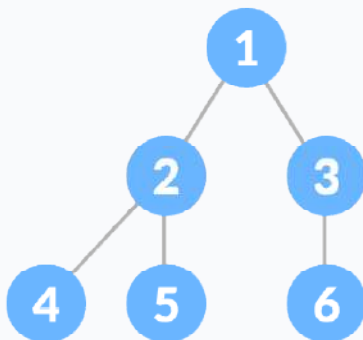


Perfect Binary Tree

3. COMPLETE BINARY TREE

A complete binary tree is just like a full binary tree, but with two major differences

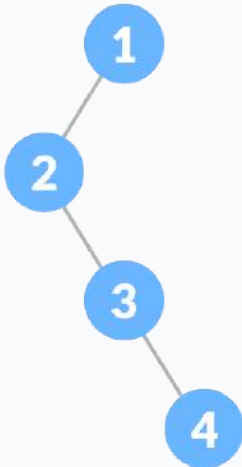
1. Every level must be completely filled
2. All the leaf elements must lean towards the left.
3. The last leaf element might not have a right sibling i.e. a complete binary tree doesn't have to be a full binary tree.



Complete Binary Tree

4. DEGENERATE OR PATHOLOGICAL TREE

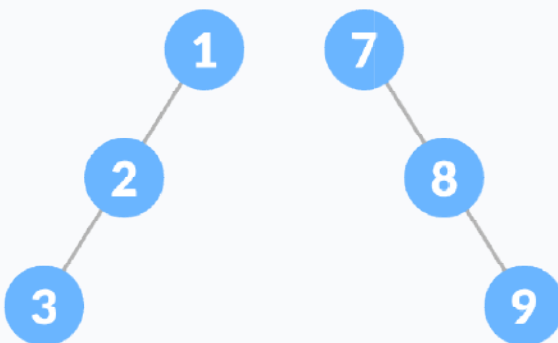
A degenerate or pathological tree is the tree having a single child either left or right.



Degenerate Binary Tree

5. SKEWED BINARY TREE

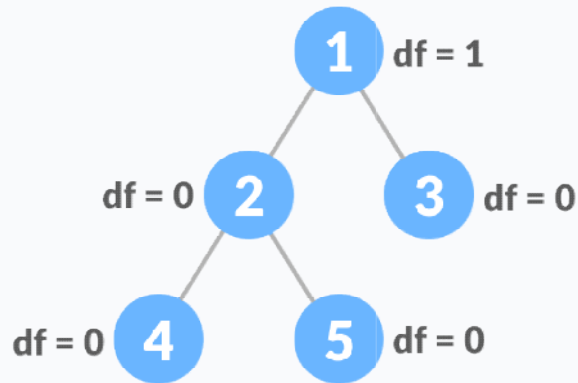
A skewed binary tree is a pathological/degenerate tree in which the tree is either dominated by the left nodes or the right nodes. Thus, there are two types of skewed binary tree: **left-skewed binary tree** and **right-skewed binary tree**.



Skewed Binary Tree

6. BALANCED BINARY TREE

It is a type of binary tree in which the difference between the height of the left and the right subtree for each node is either 0 or 1.



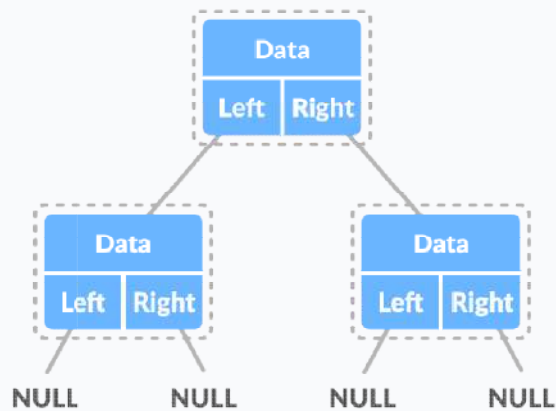
Balanced Binary Tree

To learn more, please visit [balanced binary tree](#).

BINARY TREE REPRESENTATION

A node of a binary tree is represented by a structure containing a data part and two pointers to other structures of the same type.

```
struct node
{
    int data;
    struct node *left;
    struct node *right;
};
```



Binary Tree Representation

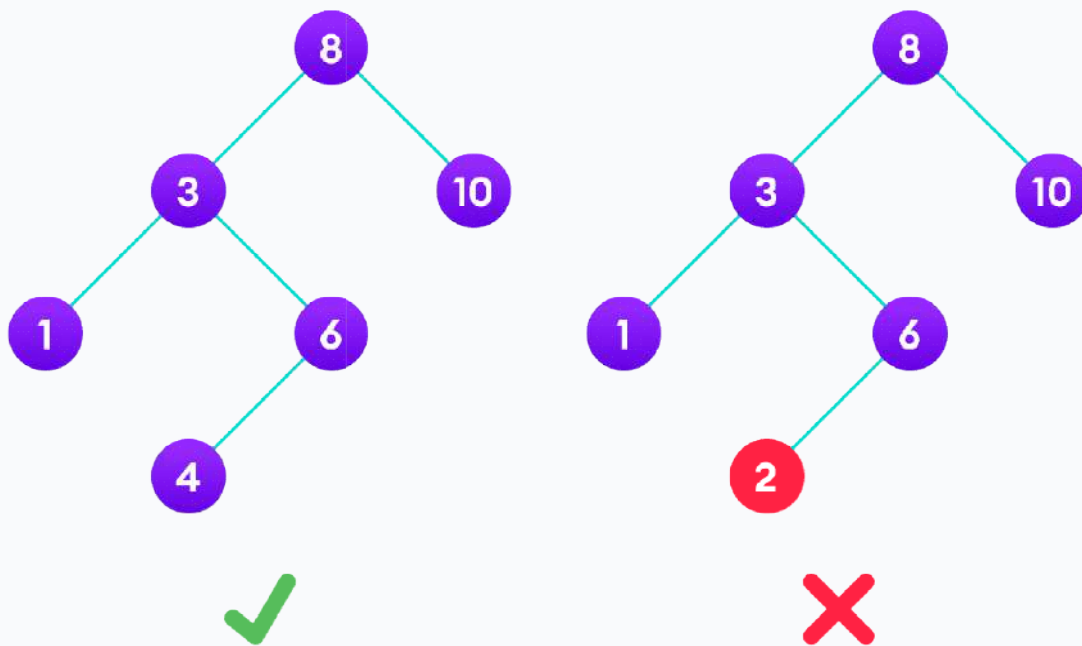
BINARY SEARCH TREE(BST)

Binary search tree is a data structure that quickly allows us to maintain a sorted list of numbers.

- It is called a binary tree because each tree node has a maximum of two children.
- It is called a search tree because it can be used to search for the presence of a number in $O(\log(n))$ time.

The properties that separate a binary search tree from a regular [binary tree](#) is

1. All nodes of left subtree are less than the root node
2. All nodes of right subtree are more than the root node
3. Both subtrees of each node are also BSTs i.e. they have the above two properties



A tree having a right subtree with one value smaller than the root is shown to demonstrate that it is not a valid binary search tree

The binary tree on the right isn't a binary search tree because the right subtree of the node "3" contains a value smaller than it.

There are two basic operations that you can perform on a binary search tree:

SEARCH OPERATION

The algorithm depends on the property of BST that if each left subtree has values below root and each right subtree has values above the root.

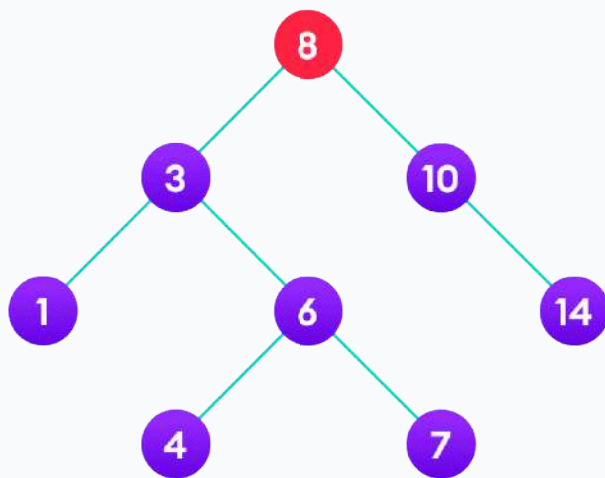
If the value is below the root, we can say for sure that the value is not in the right subtree; we need to only search in the left subtree and if the value is

above the root, we can say for sure that the value is not in the left subtree; we need to only search in the right subtree.

Algorithm:

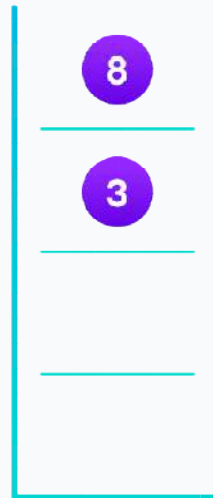
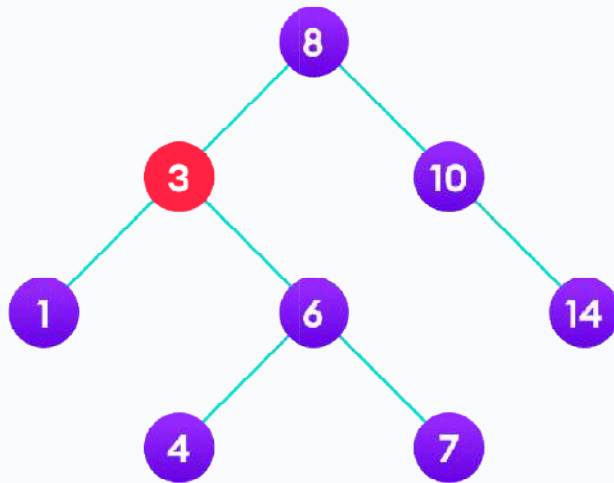
```
If root == NULL
    return NULL;
If number == root->data
    return root->data;
If number < root->data
    return search(root->left)
If number > root->data
    return search(root->right)
```

Let us try to visualize this with a diagram.



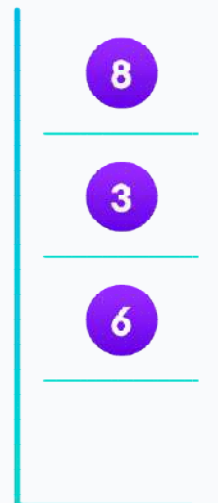
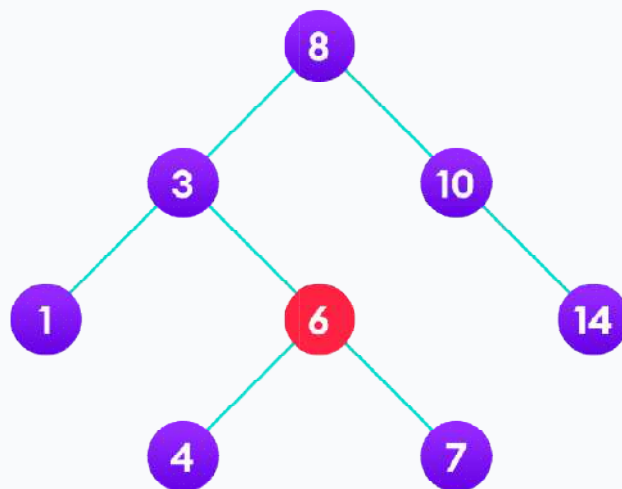
4 is

not found so, traverse through the left subtree of 8



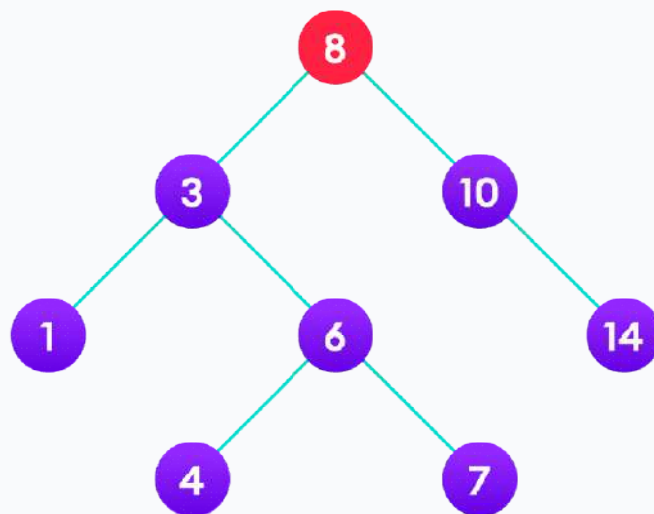
not found so, traverse through the right subtree of 3

4 is



not found so, traverse through the left subtree of 6

4 is



4 is found

Insert Operation

Inserting a value in the correct position is similar to searching because we try to maintain the rule that the left subtree is lesser than root and the right subtree is larger than root.

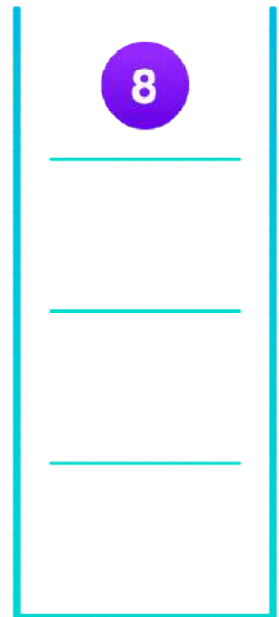
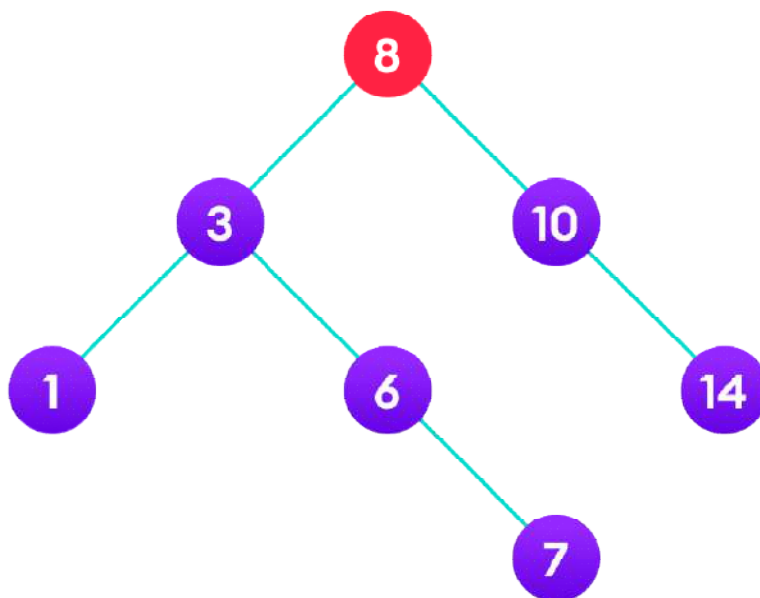
We keep going to either right subtree or left subtree depending on the value and when we reach a point left or right subtree is null, we put the new node there.

Algorithm:

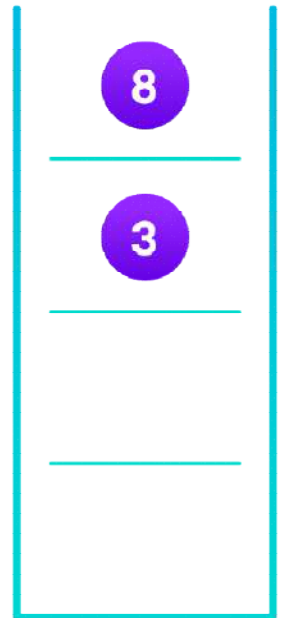
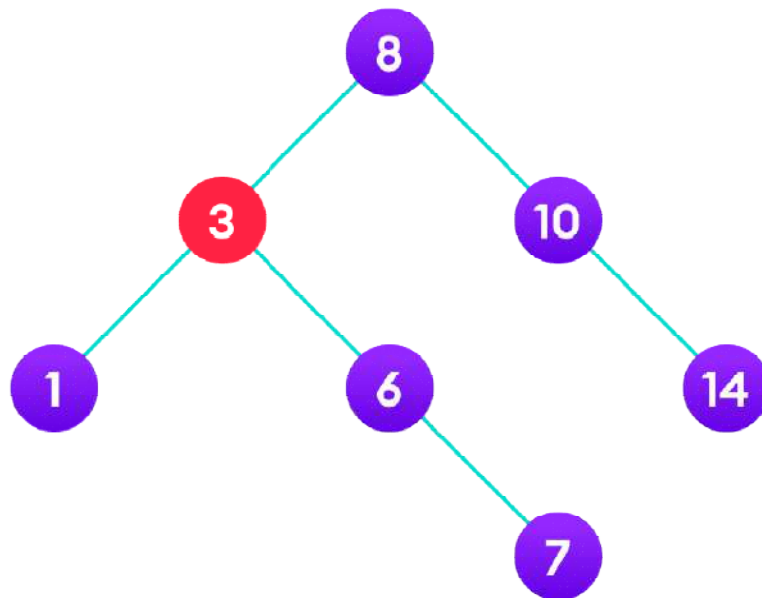
```
If node == NULL
    return createNode(data)
if (data < node->data)
    node->left = insert(node->left, data);
else if (data > node->data)
```

```
node->right = insert(node->right, data);  
return node;
```

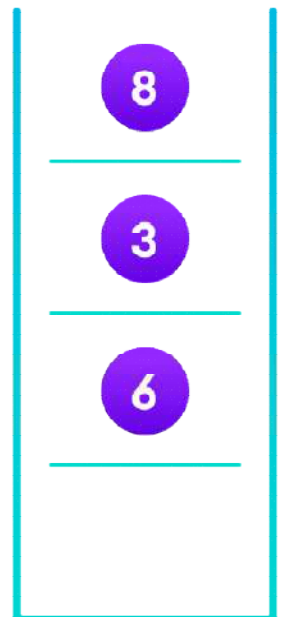
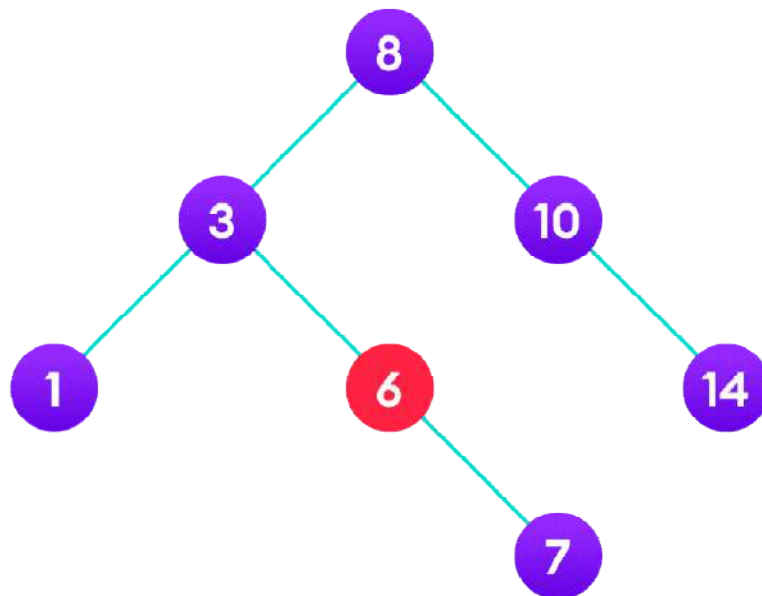
The algorithm isn't as simple as it looks. Let's try to visualize how we add a number to an existing BST.



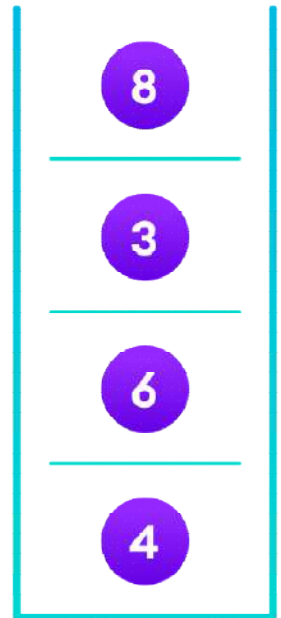
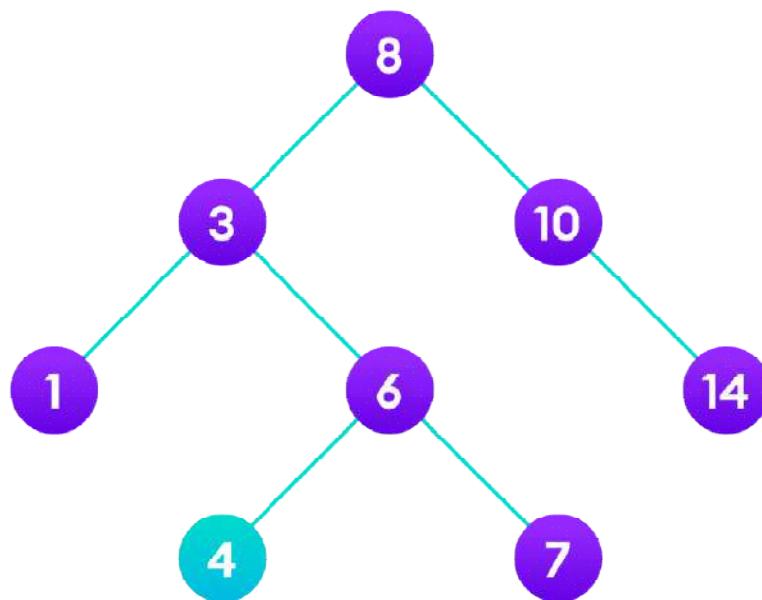
4 < 8 so, transverse through the left child of 8



4>3 so, transverse through the right child of 8



4<6 so, transverse through the left child of 6



Insert 4 as a left child of 6

We have attached the node but we still have to exit from the function without doing any damage to the rest of the tree. This is where the `return node;` at the end comes in handy. In the case of `NULL`, the newly created node is returned and attached to the parent node, otherwise the same node is returned without any change as we go up until we return to the root.

This makes sure that as we move back up the tree, the other node connections aren't changed.

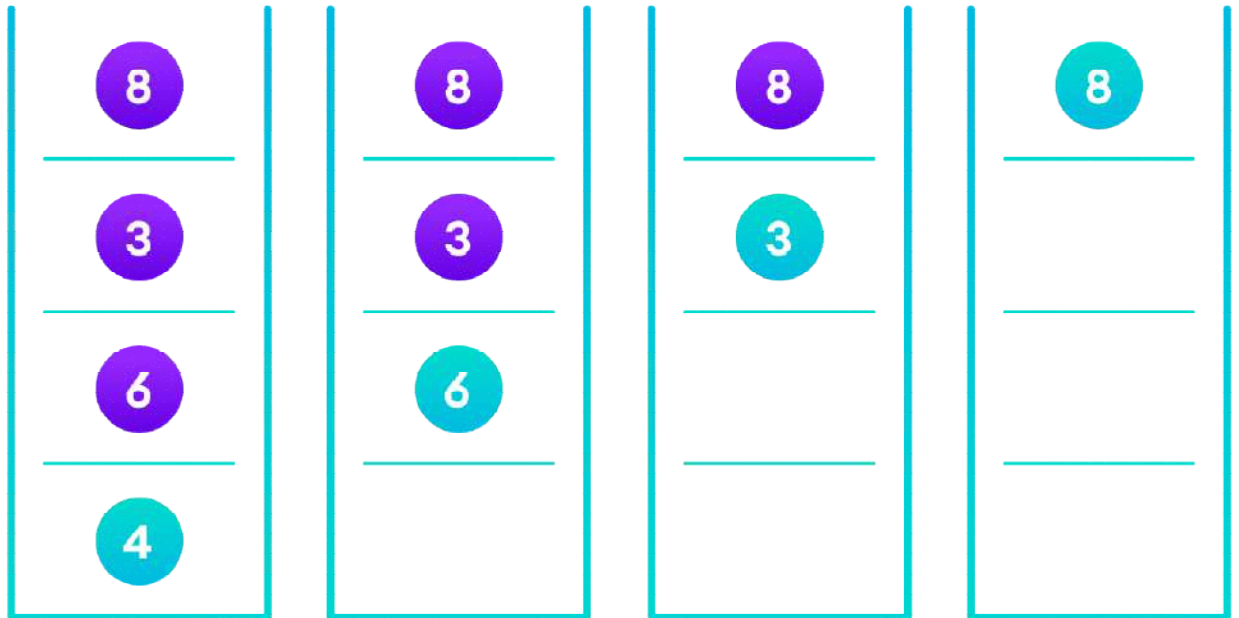


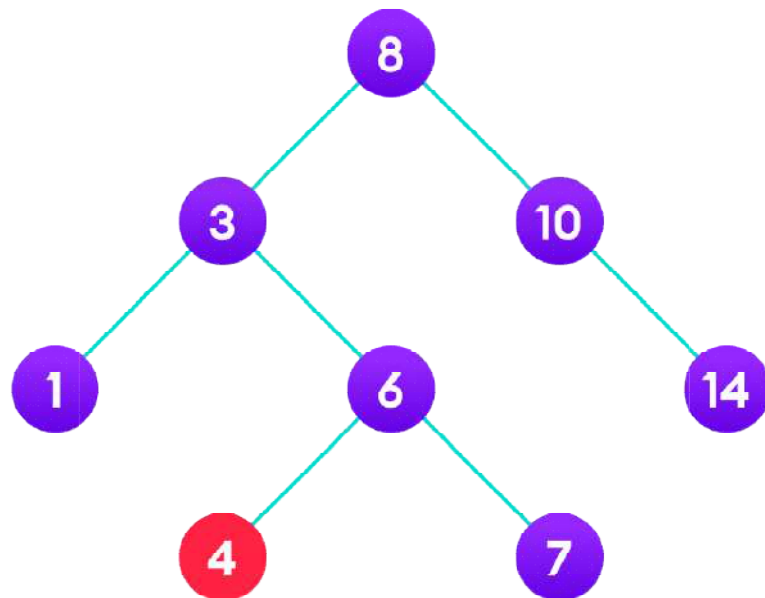
Image showing the importance of returning the root element at the end so that the elements don't lose their position during the upward recursion step.

Deletion Operation

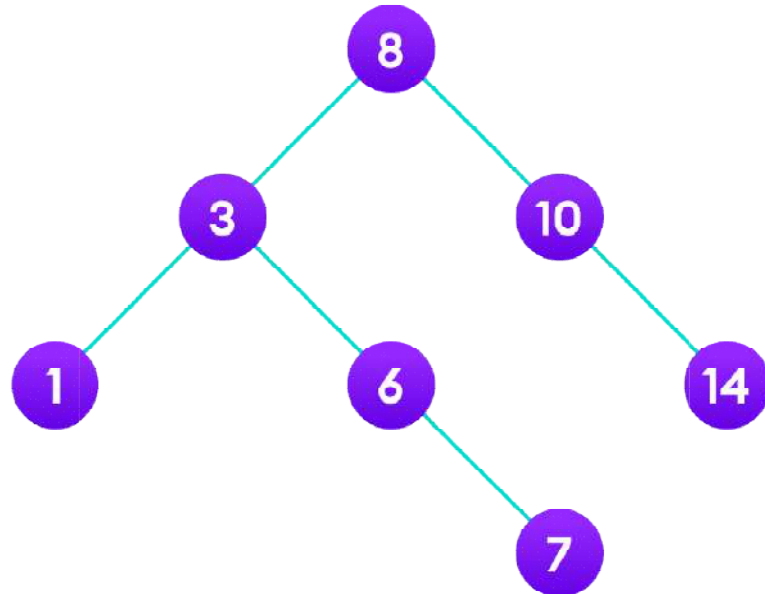
There are three cases for deleting a node from a binary search tree.

Case I

In the first case, the node to be deleted is the leaf node. In such a case, simply delete the node from the tree.



4 is to be deleted

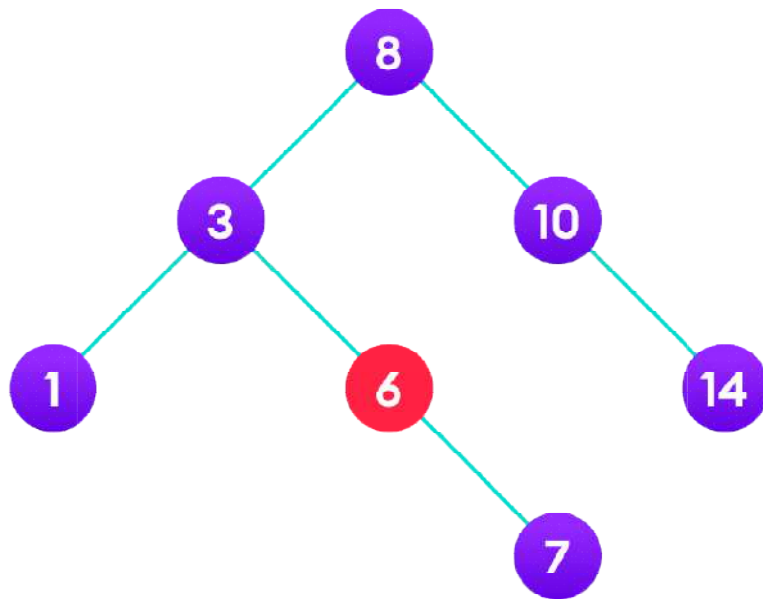


Delete the node

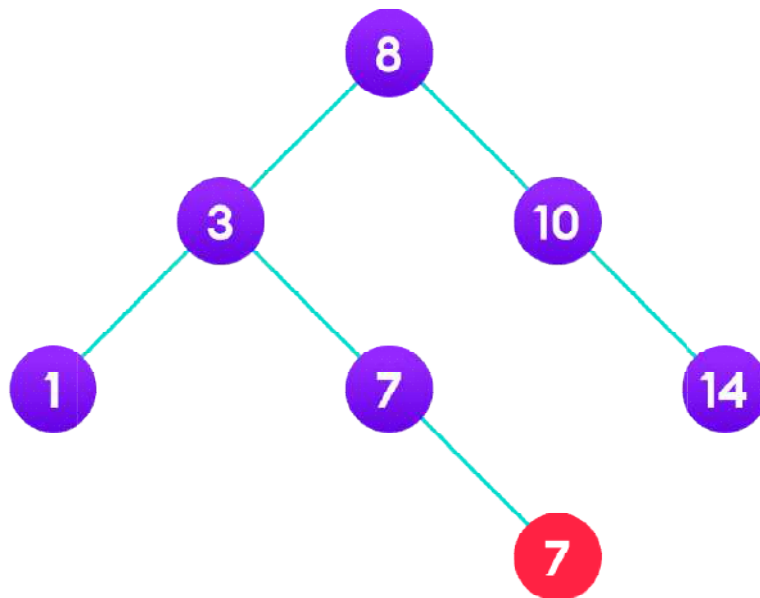
Case II

In the second case, the node to be deleted lies has a single child node. In such a case follow the steps below:

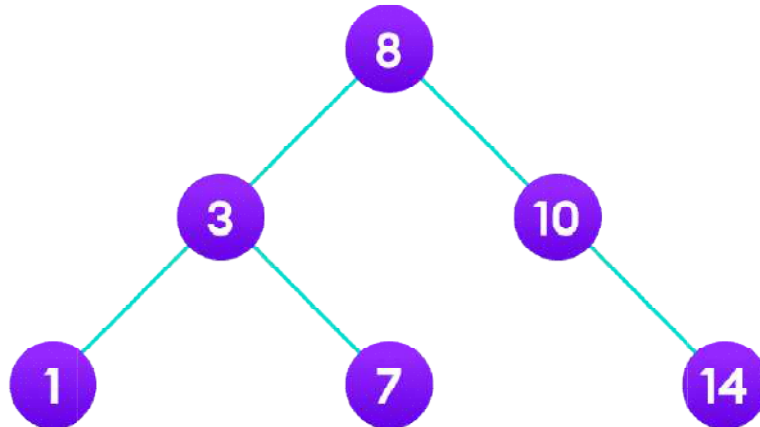
1. Replace that node with its child node.
2. Remove the child node from its original position.



6 is to be deleted



copy the value of its child to the node and delete the child



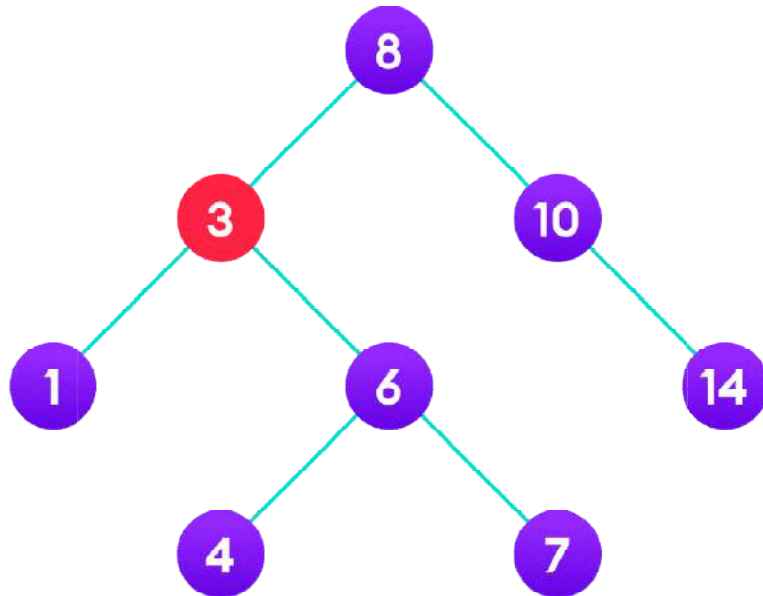
Final tree

Case III

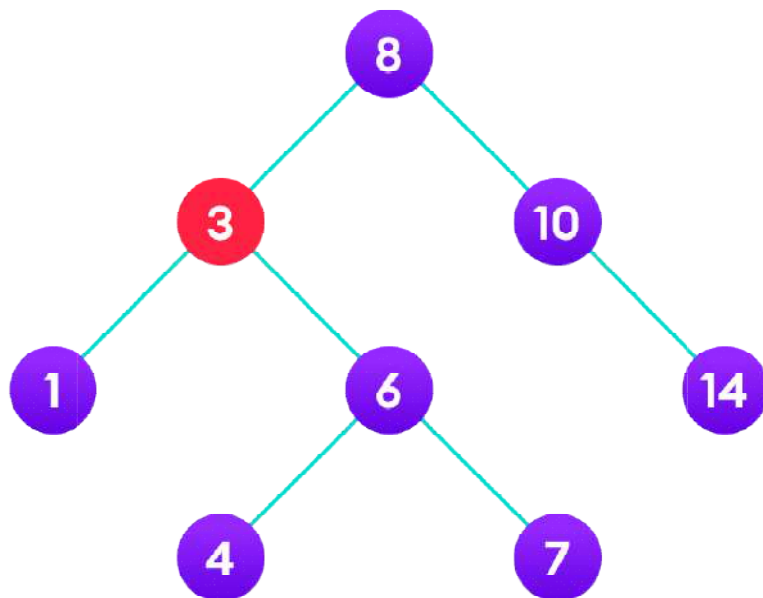
In the third case, the node to be deleted has two children. In such a case follow the steps below:

1. Get the inorder successor of that node.

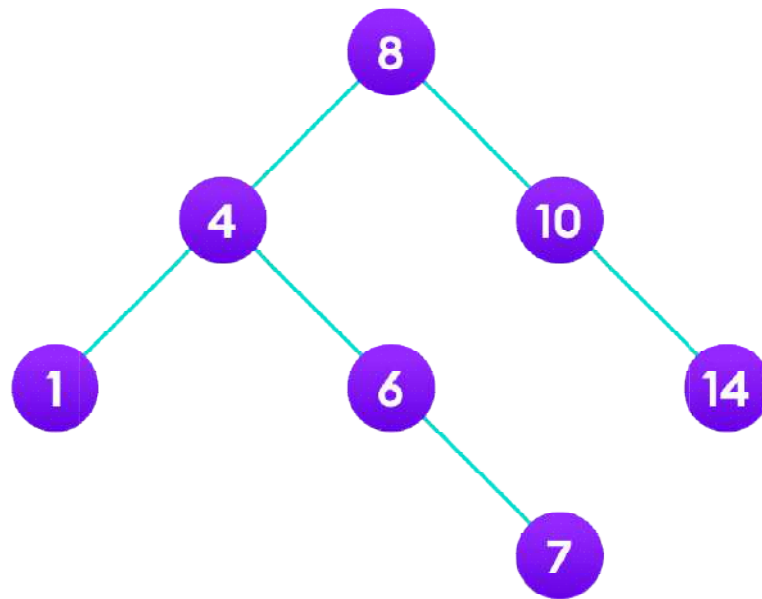
2. Replace the node with the inorder successor.
3. Remove the inorder successor from its original position.



3 is to be deleted



Copy the value of the inorder successor (4) to the node



Delete the inorder successor.
